

Part 2 - Automated Loan-Risk ML System Design

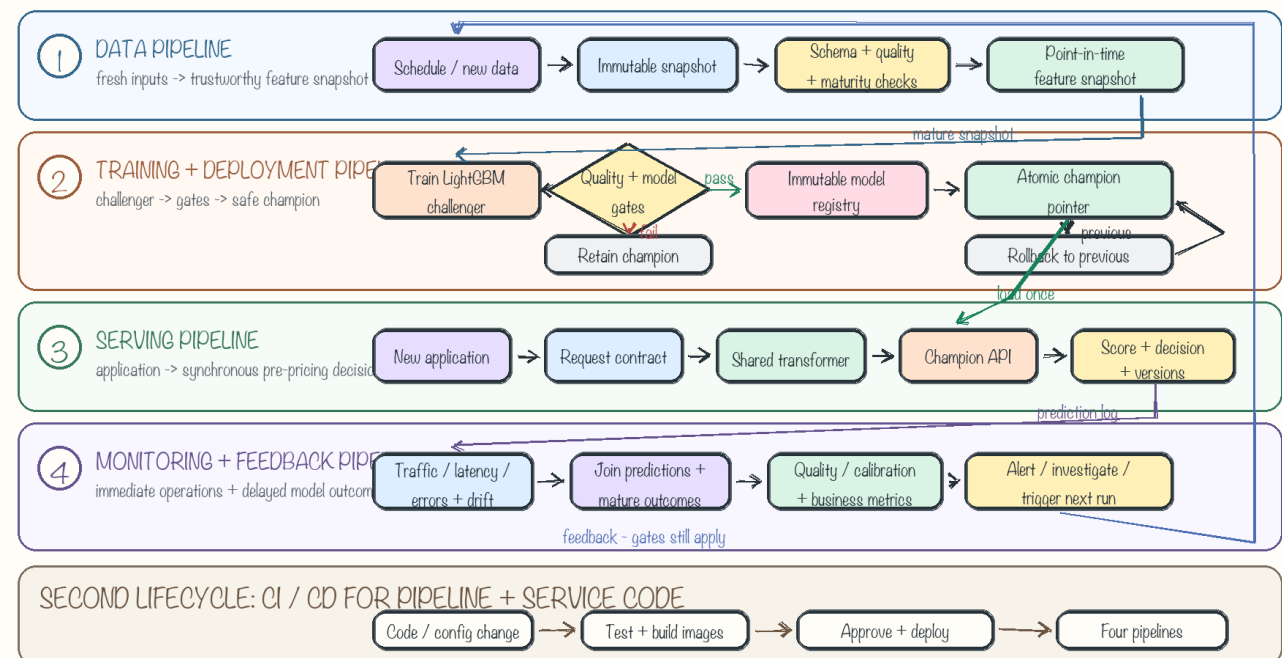
Scope. A production-oriented ML lifecycle for continuous LightGBM updates. Logical components are platform-independent; a concrete AWS deployment uses S3, container jobs/services on EKS or ECS, EventBridge scheduling, and CloudWatch/Prometheus. Part 3 preserves the same interfaces locally with Parquet snapshots, a Python orchestrator, filesystem registry, and containerised API, so it runs without cloud credentials.

Objective and operating contract

The model estimates the probability of an adverse repayment outcome for a funded-loan population. It is scored **before pricing**, so `apr` and `originallyScheduledPaymentAmount` are excluded to prevent a circular dependency. The target is 1 for Internal/External Collection, Settled Bankruptcy, or Charged Off and 0 for Paid Off Loan. Only mature, funded, unambiguous outcomes enter supervised learning; payment records are isolated from prediction-time features.

Data Scientists own modelling hypotheses, experiments, and candidate evidence. They submit a versioned bundle containing the model, ordered feature schema, configuration, proposed threshold, metrics, dependencies, and training-data cutoff. ML Engineers own reproducibility checks, serving compatibility, promotion controls, deployment, observability, and rollback. Product and risk owners approve the prediction point, target policy, business threshold, and promotion limits.

Four pipelines, two connected lifecycles



The model uses **offline batch learning**: scheduled or evidence-triggered runs train immutable challengers only from mature outcomes, and continuous training never bypasses deployment gates. New applications use **synchronous online inference** because a score is required before pricing. Stored predictions are later joined to mature outcomes for offline evaluation and the next update. Training and serving import the same deterministic feature transformer; its versioned schema records names, order, types, category handling, and prediction point, and a mismatch fails closed. Code or configuration changes follow a separate CI/CD path: unit, integration, contract, leakage, artifact, and end-to-end tests build versioned pipeline/service images before approval.

Steps and controls

- 1. Ingest and validate.** Read immutable, identified snapshots and record checksums, schemas, row counts, date ranges, and code/config versions. Enforce required types, unique non-null loan IDs, unique Clarity IDs, many-to-one joins, valid binary fields, and leakage exclusions. Quarantine contradictory terminal outcomes; fail on missing columns, duplicate keys, or incompatible schemas.
- 2. Build training data.** Generate application-time features under `pre_pricing_v1`. New applications may enter population-drift reports immediately, but become supervised examples only after an approved performance window and resolved outcome. Use rolling chronological train, validation, and untouched test windows; fit learned transformations only on training data.
- 3. Evaluate and gate.** Compare the LightGBM challenger with constant/logistic benchmarks and the deployed champion on identical rows. Hard gates cover schema, leakage, maturity, sample size, and inference compatibility. Configurable gates cover ROC-AUC, PR-AUC, log loss/calibration, recent-period stability, approved subgroup behaviour, and business value when costs exist. Use tolerances or confidence intervals for sampling noise. A failed candidate remains auditable but cannot change production.
- 4. Register and promote.** Store immutable, versioned dataset and model records containing checksums, code/config/dependency versions, source cutoffs, target and feature policies, ordered schema, threshold, metrics, and gate results. Verify checksums, reload the model, and run contract and smoke predictions before atomically changing the champion pointer. Record actor, time, reason, and previous/new versions; preserve the previous champion for rollback.
- 5. Deploy and serve safely.** Because a new application needs its score before pricing, use synchronous online inference rather than cached batch predictions. Run replicated API containers on EKS/ECS behind an internal load balancer, loading the approved champion from S3/registry once at startup; readiness fails on manifest, checksum, dependency, or contract incompatibility. Before promotion, profile feature transformation, model execution, and serialization; load-test agreed latency, throughput, memory, and error-rate limits. Responses contain probability, decision, model version, and contract version. On timeout/failure, return an explicit error so the decision system applies an approved manual/rules fallback, never implicit approval.
- 6. Monitor and improve.** Securely store prediction ID, timestamp, score/decision, model/contract versions, data-quality indicators, and latency - not raw identifiers or unrestricted payloads. Immediately monitor traffic, failures, p50/p95/p99 latency, resources, missingness, Clarity coverage, and feature/score drift. Join prediction records to access-controlled mature outcomes for discrimination, calibration, score-band outcomes, business value, time slices, and approved subgroups. Alerts prompt investigation; drift alone never promotes a model and every retraining trigger still passes all gates.

Principal limitations and decisions required

Outcomes exist only for historically funded loans, so performance cannot be established for rejected applicants and may shift when approval policy changes. The extract lacks timestamped status history, making a minimum-age maturity rule only a POC approximation. Business, risk, data, and legal owners must approve ambiguous outcomes, maturity window, threshold/costs, fairness policy, cadence, service SLOs, fallback, retention, and gate limits. Production identity, encryption, auditability, high availability, regional recovery, and infrastructure integration require environment-specific validation.